10. Fachtagung Baumaschinentechnik 2024

Automatisierung, Antriebssysteme, Bauverfahren

Towards Realistic Simulation of Construction Machines on the Example of Suction Excavator

Qi Li, Maximilian Kunz
Pankaj Deoli
Karsten Berns

Robotics Research Lab,
University of Kaiserslautern-Landau

Towards Realistic Simulation of Construction Machines on the Example of Suction Excavator

This paper presents a methodology for simulating construction machines, primarily focusing on “suction excavators” as a use case. Leveraging the capabilities of Unreal Engine 5 (UE5) [1] , including photorealistic rendering and accurate physics simulations, the methodology encompasses the vehicle and manipulator models, the integrated sensor systems, and the different scenarios involved. The effect of different disturbances on the image as well as depth quality is further experimented and analyzed for the perception tasks. The findings highlight the importance of realistic simulations towards intelligent decision-making for the use-case of Construction machine development.

1 Introduction

In modern construction industry, the demand for efficiency, precision and safety is significant, particularly when it comes to the operation of specialized machinery. Among these, construction machines with extended arms (e.g. suction excavators), are essential for tasks like digging, cleaning, and debris removal. Such machines are relied upon to perform accurately under a variety of conditions, making their precise control and operation a critical focus for the industry. They typically consist of a commercial vehicle platform and a hydraulic manipulator mounted on it [1]. On one hand, due to this versatility, they serve various industries including construction, landscaping, civil engineering, and forestry. On the other hand, operating these machines requires skilled operators due to the physical challenges and the working conditions involved. To this end, there is a growing demand for fully autonomous construction machines to improve process quality, safety, and cost-effectiveness. However, automating these systems is challenging due to their unstructured working environments, often requiring extensive design, testing, and validation efforts.
Traditional approaches often involve physical prototypes or field tests, which can be insanely expensive and time-consuming. More-over, the inherent dangers associated with operating heavy machinery in uncontrolled environments pose safety risks, making direct experimentation impractical for extensive testing and iterative development. To mitigate these challenges, there is a growing need for alternative methods that can accurately replicate the behaviour of construction machines in a controlled and safe manner. Therefore, leveraging realistic simulations within UE5 offers a viable solution, enabling early testing, design iteration, and sensor optimization, thereby bridging the gap between simulation and reality [2] [3] [4] [5]. An example of this can be seen in Figure 1 wherein (a) showcases a real suctions excavator and (b) a simulated excavator in UE5 respectively.

2 Related Work

Simulation environments are essential for accelerating the development, evaluation, and testing of robotic systems [2]. Lightweight platforms like CopelliaSim, SimVis3D, and

[1] https://www.unrealengine.com/de/unreal-engine-5



Figure 1: An overview of suction scenarios in real-world vs simulated environment. Left: Real world suction application scenario. Image from [6], right: Simulated suction scenario in UE5

Gazebo offer fundamental features such as physics engines, basic 3D rendering, and built-in controllers and sensors, often integrating with the Robot Operating System (ROS) [7]. Since approaches such as reinforcement learning require extensive data sampling from simulation environments, a lightweight setup is often preferred to efficiently generate dense datasets. Examples of such lightweight environments include Mujoco, PyBullet, and Gazebo [8]. These environments are effective for many robotic applications but have limitations when dealing with large, complex environments and achieving photo-realistic textures.

For several years, game engines like Unity or UE5 have been used for simulating various robot-related tasks [2]. Frameworks like CARLA [9] or AirSim [3] have been developed within the UE5 to support and simplify autonomous driving. They utilize the broad range a gaming engine provides for creating street networks, buildings, weather effects or pedestrian simulations. An example of the rich data that can be acquired from CARLA can be seen in Figure 2.

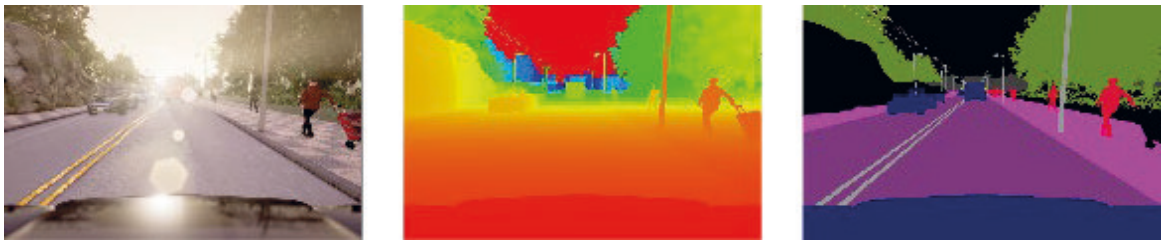


Figure 2: Example sensor output from the CARLA simulation. **Left**: image of a mono camera, **middle**: depth image in RGB colors, **right**: semantic segmentation. The last two sensors are called pseudo-sensors since they do not have a direct representation in the physical world. Image from [9, page 3]

Modeling a construction machine, specifically a suction excavator in UE5, requires four key components: the interface between UE5 and our robot control framework Finroc [10], vehicle and manipulator models, control components, and sensor systems. These components are discussed in [2], though their work is based on Unreal Engine 4 and does not include newer features like the Chaos Vehicle system. Additionally, the manipulator kinematics in [2] rely on an analytical model, which is not ideal for high-degree-of-freedom (DoF) manipulators. To address this, we use a more common robot description format, URDF [11], and an open-source kinematic library to solve the kinematics. The sensor system described in [2] has also been restructured by [12].

Utilizing the open-source format of USD (Universal Scene Description) Nvidia's Omniverse[2] aims at connecting different software applications in one virtual world. Their goal is to create an intuitive simulation workflow (e.g. for digital twins and AI systems) using state of the art rendering techniques. For robot applications, Nvidia has developed Isaac Sim[3], an application built inside Omniverse for testing, developing and training ai-based robots featuring a physically accurate environment. Isaac Sim also features a lightweight simulator specialized for robot learning called Isaac Lab[4] [13][14].

3 Construction Machine Modeling

In this section, the modelling process of the construction machine on the example of a suction excavator (including different tools and dependencies) will be introduced. Figure 3 shows the important components of our workflow for simulating robots. The main component is the Unreal Editor (center), where our robot interacts with the environment.
For controlling the robot from outside, we use our robotic framework Finroc. Besides sending control commands, we also receive sensor input from UE5 like camera images and point clouds. Those are forwarded to Finroc where the current state of the robot and its environment is updated. After that, a new control commands for the next cycle are transmitted to UE5. For modeling the vehicle and arms, we use the open-source 3D modeling software Blender[5].

3.1 Vehicle Modeling

Our vehicle has 4 axles i.e. one steering front axle, one steered leading axle and two driven axles. The digital model of the vehicle is commonly provided by the manufacturer or their partners in a common CAD format (e.g. .step, .obj, .stl). The first step is always to simplify the model since the number of vertices is usually too high to work with in Unreal and there are many unnecessary details (e.g. screws, engines or other parts of the machine invisible from the outside). As mentioned previously, we use Blender to accomplish this task. After the model simplification, it has to be modified for importing into the Unreal Editor. This includes (among other things), separating the moving parts for animation (wheels, joints, links) from the main vehicle body, the chassis. As a last step, textures are applied to the correct areas for a visually satisfying representation of the vehicle. After that, the model can be exported as a .fbx file.
In UE5, the Chaos Vehicle Plugin [15] is used to model the kinematic of the vehicle. It provides an Ackermann model for steering, and a set of parameters like suspension settings and mass distributions. For this, we import the previously mentioned .fbx file which is interpreted as a skeletal mesh and a physics asset in UE5 (see figure 3). Besides the collision, we also need the animation of the wheels which is done in an animation blueprint

[2] https://www.nvidia.com/en-us/omniverse/

[3] https://developer.nvidia.com/isaac/sim

[4] https://isaac-sim.github.io/IsaacLab/

[5] https://www.blender.org/

already provided by the editor. Together with the wheel setup, the chaos vehicle movement component manages the animation of a simple vehicle. For more detailed instructions of the Chaos Plugin, the official documentation[6] should be used. In the end, we have one actor (BP MTS Truck) which is the digital representation of the physical vehicle with its sensors

[6] https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-set-up-vehicles-in-
unreal-engine

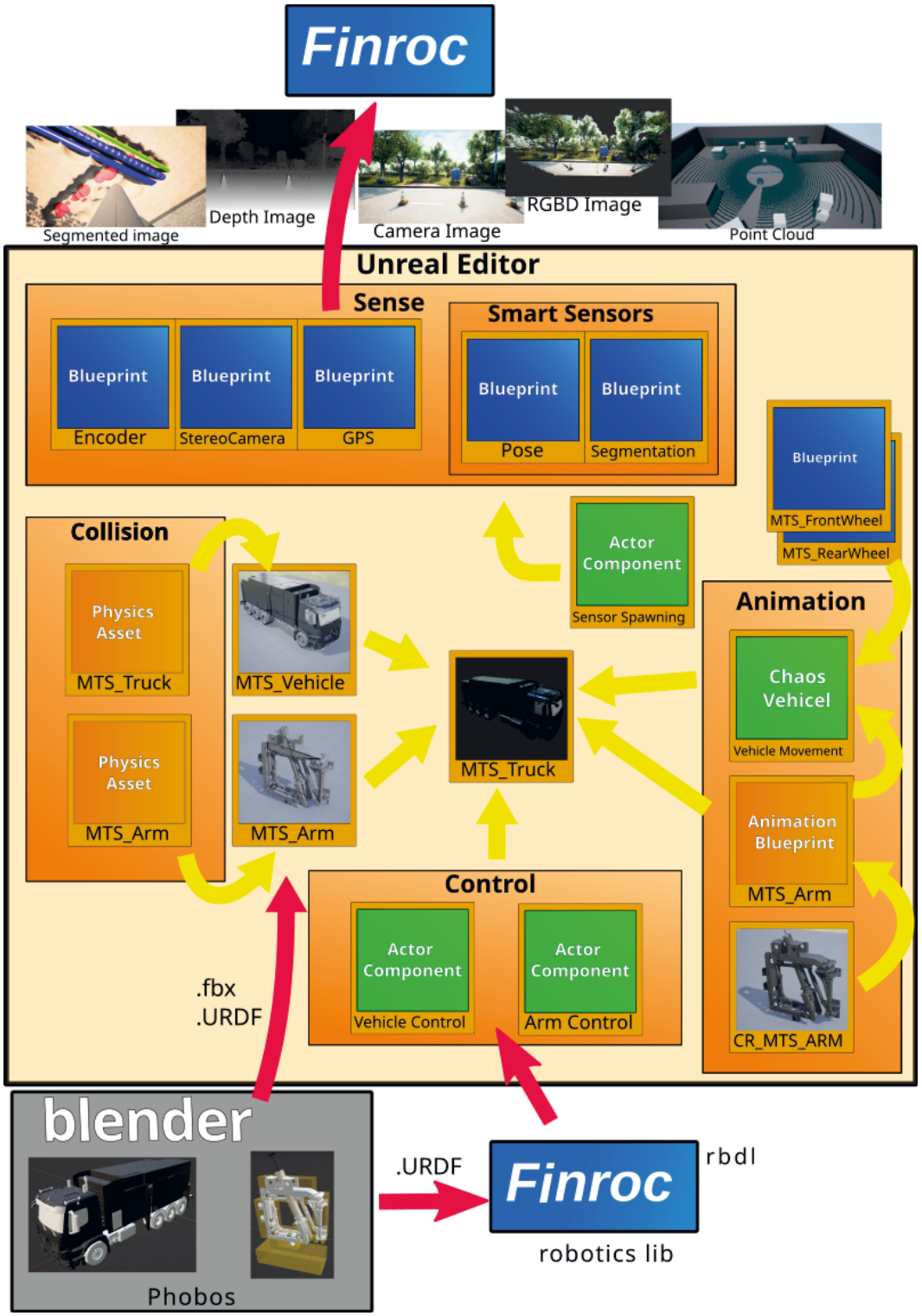


Figure 3: Overview of our workflow from CAD files to controlling a robot using its sensor input. The central part is the Unreal Editor where we have a digital model of our robot sending sensor data to our robot framework Finroc. Those inputs are used to create a control command which is sent via our Unreal-Finroc interface.

and kinematic settings. The control commands from Finroc can be received by the UE5 through a port-like communication explained in [2]. We additionally have the option to transmit commands via UDP or TCP as mentioned in [12].

3.2 Manipulator Modeling

Modeling of the manipulator can be divided into two parts: first the kinematic model and second animating the model in UE5. The original manipulator uses hydraulic based cylinder to move its joints and attachments. For simplification, we only model the joints movement directly, ignoring the behavior of hydraulic cylinders. Since the URDF from ROS is supported by the most kinematic and dynamic libraries, we choose URDF format to model the manipulator. Rather than building the DH parameter table [16] and then modelling the manipulator according to the URDF by hand, we use Phobos [17] to model the kinematics. Phobos is a Blender plugin which allows the user to define the link and joint parameters visually, also providing the resulting .urdf file. Like the preprocessing for the vehicle (as explained in subsection 3.1), the 3D model of the robot arm also needs modifications. All links and joints have to be separated and their coordinate system has to be updated to follow the DH convention [16]. Doing that with visual feedback is much more convenient and leads to a smoother workflow. The import in UE5 is similar to the previously mentioned vehicle model with the only exception that there is no pre-existing animation provided by the editor. By using Control Rigs[7] we can animate the single joint movements using a forward solve graph. Additionally, an inverse kinematic chain can be used for the movement. For controlling the arm, we use the same communication interface as we do for the vehicle. In Finroc, we integrated multiple kinematic libs: robotics lib [18], rbdl [19], which can perform forward and inverse kinematics. They are based on the URDF file, which we previously exported from Blender. The final setup can be seen in figure 4.



Figure 4: Setup of the suction excavator in UE5 with suction arm and the vehicle (MTS).

3.3 Suction Scenario Modeling

The main working areas of the suction excavator are urban environments. There, the task is to remove materials like sand, soil or gravel without damaging cables, pipes or roots of trees. Obstacles above the ground could include trees, fences, walls, traffic lights, pillars and so on. For simulating this typical suction scenario, we first need an urban environment, including obstacles and our objects of interest (cables, pipes, roots). Last but not least, we need to simulate the material change of the suction process.

[7] https://dev.epicgames.com/documentation/en-us/unreal-engine/control-rig-in-unreal-engine

Luckily, UE5 offers a store[8] featuring many assets for free. We chose the city park environment for our typical urban scenario which can be seen in Figure 5. For our obstacles and objects of interest we also used free assets from the store. To simulate the material changing process of the suction operation, the surface of the working area should be changeable during runtime. In UE5, the landscape system is based on the height map and mainly suited for static design phase and not for manipulation during runtime. Another possibility would include procedurally generated meshes which would have to be created by changing the vertices of a plane according to the suction process. To reduce the workload, we decided to use the Voxel Plugin[9] to perform the task of surface manipulation. Since the plugin is based on a 3D voxel representation, it is able to generate complex 3D surfaces and supports overhangs during runtime. Different ways for manipulating the surface are also included. Figure 5 shows a complex suction scenario including obstacles, our objects of interest and the voxel surface in the urban environment. On the left, an overview of the scene can be seen with the vehicle starting the suction process.



Figure 5: Example of a suction scenario in a simulated urban environment before the process (left) with a view underground (right).

4 Sensor Modeling

This section deals with the modeling of our stereo camera as well as the smart sensors we used for the suction excavator. Additionally, we shortly talk about image effects such as lens flares which we applied as noise on our cameras to check for robustness.

4.1 Stereo Camera and Lens Effects Modeling

For our virtual stereo camera, we mainly relied upon our implementation of a RGB camera with additional settings. By doing so, we can also use the existing effects we have applied on mono cameras to the respective cameras of the stereo camera.

[8] https://www.unrealengine.com/marketplace/en-US/store

[9] https://voxelplugin.com

4.1.1 Stereo Camera

The real stereo camera uses inbuilt stereo matching algorithm to get the depth information also generating the point cloud. In simulation we should also use the same algorithms to generate the point cloud to narrow the domain gap between real and simulation. According to [20], stereo camera is a real solid-state sensor for 3D sensing, it can also provide reach color and texture information with usually 15-20 Hz frequency. It is also very cheap in comparison to LiDAR and therefore is very suitable for commercial autonomous applications. Unlike [2], where the authors only output the image vector then do the stereo matching in the robot control system, we implemented the stereo camera in unreal engine makes it can direct output point cloud and depth image, this allows the sensor sensing independent on the robot control system. Our stereo sensor uses two already implemented mono cameras with a fixed offset returning the left and right RGB image, the disparity image and a point cloud with or without the RGB values. The stereo camera is inherited from the SensorParent class introduced in [13], which handles port creation for communication between UE5 and Finroc. The stereo matching algorithm is based on OpenCV and libSGM[10]. OpenCV provides batch matching and semi-global matching algorithms and can be included in the UE5 via a plugin. On a negative note, CUDA is not directly supported by UE5, but by utilizing libSGM we can use CUDA to perform semi-global matching efficiently. After we get the disparity map from stereo matching, we can re-project it to point cloud based on Equation 3 from [20] using a method provided by OpenCV.

$$\begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{pmatrix} = Q \cdot \begin{pmatrix} u \\ v \\ D_{(u,v)} \\ 1 \end{pmatrix} = \begin{pmatrix} u - c_x \\ v - c_y \\ f \\ \frac{D_{(u,v)}}{b} \end{pmatrix} \tag{1}$$

$$Q = \begin{pmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & \frac{1}{b} & 0 \end{pmatrix} \tag{2}$$

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix}^{(CCS)} = \begin{pmatrix} \frac{w_0}{w_3} \\ \frac{w_1}{w_3} \\ \frac{w_2}{w_3} \end{pmatrix} \tag{3}$$

where, f is focal length in pixel units of left and right image plane, b is the baseline of the stereo camera, (x, y, z) is the 3D coordinate of a point in the camera coordinate system, (u, v) is the pixel coordinate of the image in the image coordinate system (ICS), $D(u, v)$ is the disparity value, $(c_x,\ c_y)$is the principal point which is, in an ideal case, the center of the image plane. The output of the stereo camera is shown in Figure 6.

[10] https://github.com/fixstars/libSGM



Figure 6: Simulated output of the stereo camera: RGB image, depth image and point cloud with RGB information

4.1.2 Lens Effects

In the real world, there are a lot of disturbances which can affect the sensor's output. To narrow the gap between the simulation and reality, these disturbances should also be modeled. On a construction site, one might see one or more of the following ones: vibration, lens flare, over/underexposure, dirt and rain drops.
For vibration effect we used a compact 6 DoF motion platform [21] for which we have a digital representation in UE5 with the ability to transmit the movement. Thus, by placing a stereo sensor on the physical platform as well as on the virtual in simulation, we can see mirrored movement in the stereo camera influencing the matching and detection.
The other disturbances are implemented using post process settings allowing for manipulation of the image using shaders. Many effects that are commonly used in virtual production or filming are already usable by default. Figure 9 shows these lens effects in UE5. Noise directly on the lens are done using different masks and the rain drop effect was done using a shader.

4.2 Virtual Smart Sensor Modeling

As mentioned in 3, we have used some virtual sensors that do not have a direct representation in the physical world, called smart or pseudo sensors. An example of such a sensor can be seen in Figure 8. The smart sensor used there is a camera that does the segmentation of arbitrary objects (in our case pipes, roots and boulders) in simulation. Utilizing these smart sensors has two main advantages. Firstly, we can quickly test which objects could be of interest for segmentation or object detection. Secondly, we can improve our workflow and work in parallel in a team. In the case of the previously mentioned smart sensors for segmentation, we also use it for generating training and test data for neural networks. Other smart sensors include a pose actor returning the accurate 3D pose from the simulation as ground truth for localization or detection of arbitrary objects for faster prototyping.

5 Experiments

In this chapter, a comprehensive testing process for the previously introduced modules is conducted. The trajectory tracking aims to test whether the manipulator model is controllable, while the suction scenario testing evaluates the suction process. The

disturbances of the stereo camera are tested in both real and simulated scenarios to determine whether the simulated lens effects perform similarly to the real effects.

5.1 Trajectory Tracking

The goal of this experiment is to test whether the manipulator model in UE5 is controllable through Finroc. To achieve this, we defined a complex spline that covers most parts of the manipulator's workspace. The trajectories are passed to the controller through the communication port between Unreal and Finroc. Since the manipulator in UE5 only supports position control, the controller contains only an IK (Inverse Kinematics) solver based on the rbdl library and does not include any optimization. The input TCP pose is taken, and the corresponding joint configurations are output by the IK solver. These joint configurations are then sent to the manipulator as the target positions. The results are shown in Figure 7. The figure demonstrates that the manipulator is controllable through position control. Despite the absence of a lower PID controller and optimization in the IK solver, the MSE error remains acceptable at 0.025 meters.

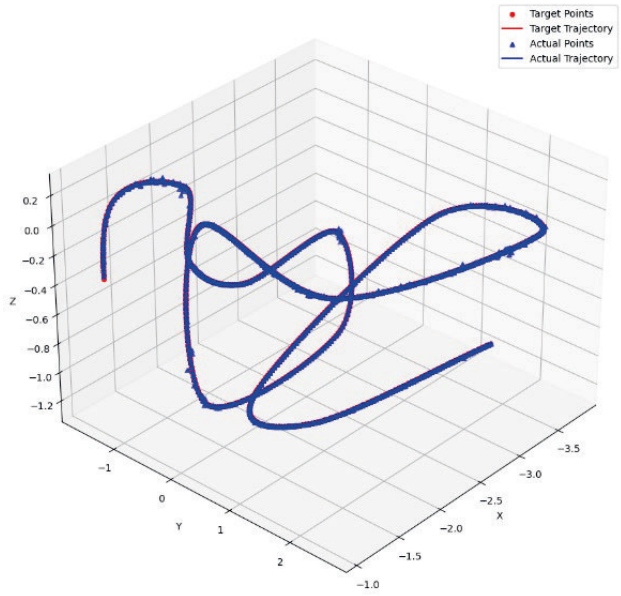


Figure 7: The trajectory of the manipulator in its workspace. The red line represents the target trajectory, and the blue line represents the actual trajectory. The mean error between these two trajectories is 0.025 meters.

5.2 Suction Process

The suction process is primarily based on the Voxel Plugin and objects placed underground. We set the voxel world size to 256 and the voxel size to 6 cm for better performance. In this experiment, we use the IK solver on the Finroc side to implement a TCP (Tool Center Point) controller, as described in subsection 5.1. We define a simple spline for the manipulator to follow, positioning the TCP near the voxel world to remove material. The diameter of the removed sphere in the voxel world is set to match the diameter of the suction pipe. A camera is mounted on the manipulator to record the process, as shown in Figure 8. The results indicate that the Voxel Plugin successfully simulates the complex changes in the pit surface.

The interaction between the underground objects and the voxel world is also realistic, as the material is gradually removed and the objects are slowly exposed. Additionally, we tested a smart sensor with a segmentation component. The results show that the smart sensor successfully segments all the objects exposed to the air.

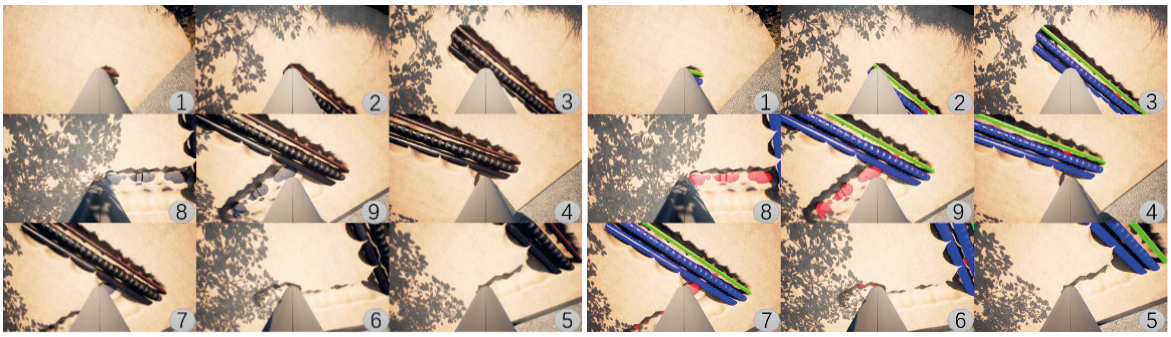


Figure 8: Suction process with and without the segmentation component.

5.3 Disturbances on Simulated Stereo Camera

Proper simulation of the expected disturbances in real-world applications is of utmost importance when working with construction machines. To this extent, we simulated and tested various effects, including lens effects such as overexposure, underexposure, bloom, lens flare, dirt masks, raindrops, and vibrations, on both simulated and real stereo cameras.

5.3.1 Lens Effects

To study the impact of lens disturbances on the stereo camera, we applied various effects as described in section 4.1 in both the simulation and the ZED2 stereo camera. The simulated and real ZED2 cameras were set to the same parameters: resolution of $1280 * 720$, a baseline of $12mm$ and a DFOV of 118°.The scenario was also modeled in UE5 to minimize environmental disturbances. For the lens effects—dirt mask, raindrop, and lens flare (with bloom)—we recorded the color and depth images for both the real and simulated scenarios, with and without the lens effects. The results shown in Figure 9. In these comparisons, the scenarios with lens effects demonstrate that the color and depth images in both the real and simulated scenarios are similar. After the lens effects were applied, the simulated lens effects produced the same impact on the color and depth images as the real lens effects. Notably, in the depth images, the affected region's depth values were reduced to 0, indicating that the stereo matching algorithm failed to find corresponding pixels in the left and right images of the stereo camera. This experiment demonstrates that the simulated stereo camera performs similarly to the real one, and we can effectively simulate lens effects to achieve realism comparable to real-world conditions.

5.3.2 Vibrations

In the vibration experiment we applied different vibration as discussed in section 4.1 and recorded the data in both real and simulated stereo camera which attached on the platform. Initially, we hypothesized that the stereo matching process and the object detection

algorithm (YOLO-V8[11] in our case) would experience performance issues due to vibrations (inherently) present in the operational environment. Surprisingly, the results indicated that vibrations did not have a significant effect on both stereo matching process and also the object detection algorithm's performance thereby indicating the robustness of the model.

However, it is important to know the limitations of our experimental setup. We only applied low frequency and amplitude to the vibration platform, which cannot fully represent the complexity or intensity of vibrations present in the actual operating environment. Our "controlled" laboratory conditions cannot capture the full spectrum of variables and noises that might occur in real-world scenarios.

Given these considerations, the current findings should be viewed as preliminary. The absence of noticeable effects in our experiments does not conclusively remove the potential impact of vibrations on YOLO-V8 under "all" conditions.



Figure 9: This figure presents the results of the lens effect experiment. Three lens effects are displayed from top to bottom: dirt mask, lens flare with bloom, and raindrop. For each effect, the upper row shows the recordings from the real ZED2 camera, and the lower row shows the recordings

[11] https://github.com/ultralytics/ultralytics

from the simulated stereo camera. In each recording, the left side displays the images without the lens effect, while the right side displays the images with the lens effect applied.

5.4 Image processing pipeline

In order to examine the effects of disturbances (as explained before) on the image processing pipeline, different object detection strategies were employed. Since the application scenarios varies from urban city environments to areas with trees, two different models of YOLO-V8 were experimented with.

For general objects in urban scenarios, YOLO-V8 (pre-trained on COCO) was used. The backbone (mAP (53.9), 68.2M parameters) is the slowest in speed but very accurate in detection performance. The experiment was done in simulation as well as with real images. The results of it are shown in Figure 10.

For the case of trees, the YOLO-V8 (pre-trained on COCO) was fine-tuned with SYNTH43k tree dataset [22]. The model was trained for 50 epochs with a batch size of 8 on Nvidia GeForce 3080Ti (6 GB). The results are shown in figure 10. As visible, under intense sun flare (or overexposure), the performance of the model decreased significantly. Because of overexposure, the object's distinctiveness was hampered thereby leading to poor performance.

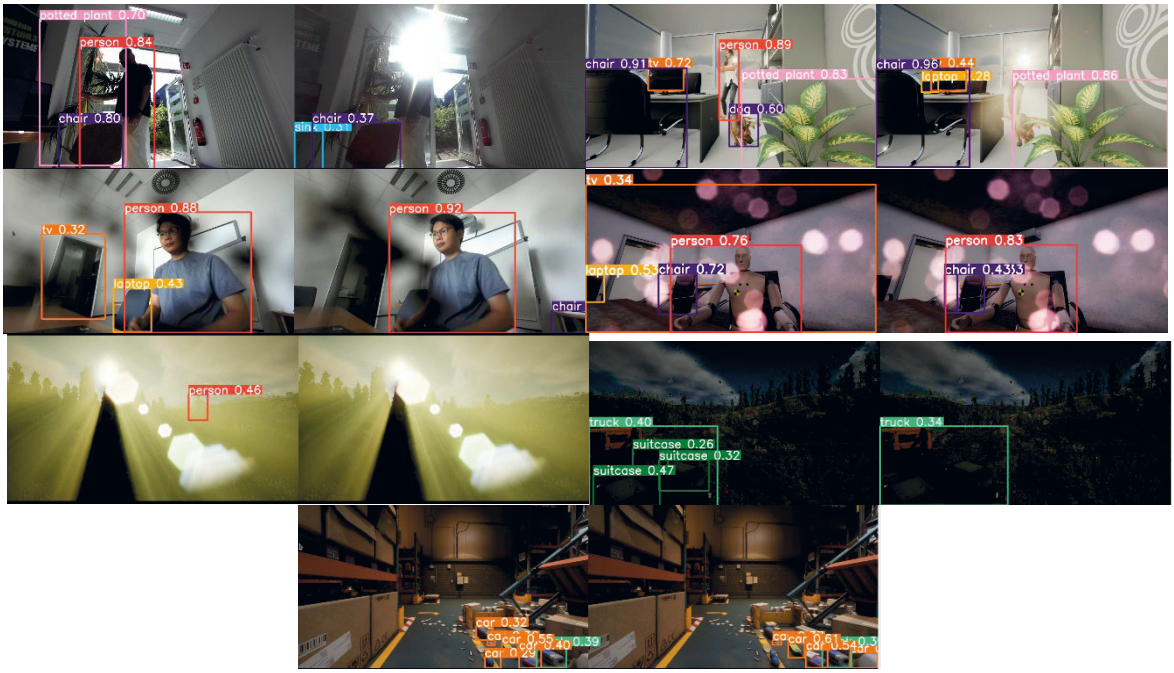


Figure 10: Effect of various disturbances on the performance of YOLO-V8

6 Conclusion

In this work, we presented a method that leverages the capabilities of Unreal Engine to realistically model construction machinery, sensor systems, and their disturbances. Using Blender, we simplified and modeled the vehicle and manipulator, exporting them in FBX format and generating the URDF description of the manipulator. The suction process for the suction excavator was modeled with the help of the Voxel Plugin in Unreal Engine. We

further developed the vehicle and manipulator models in Unreal Engine and integrated them with the kinematics library utilized in Finroc to make these models controllable. Additionally, we modeled the stereo camera and smart sensors, such as segmentation sensors and 3D pose sensors, as well as various disturbances affecting the stereo camera, including lens effects and vibrations.

In the experiments, we tested the manipulator's controllability, the suction process, and the disturbances affecting the stereo camera. The results demonstrate that we successfully created a realistic simulation system that can be used for the development, validation, and testing of construction machines.

Despite these achievements, there are several areas where the proposed method could be improved. As we simplified the hydraulic system and only controlled the joint positions of the manipulator, our model is not fully capable of representing a real hydraulically actuated manipulator. Proper modeling of the hydraulic system is necessary to make the simulation of construction machinery more realistic. Additionally, introducing a random scene generator to create diverse construction scenarios could enhance testing and data gathering. For the suction process, the objects beneath the surface are challenging for optical-based sensors to detect, so a contact sensor that can output contact signals and measure contact forces should be developed.

7 Acknowledgement

This work is a part of a publicly funded project with Commercial Vehicle Cluster (CVC) in collaboration with Mobile Tiefbau Saugsysteme GmbH (MTS).

Quellenverzeichnis:

[1] Varga, B.: Control of large vehicle-manipulators with human operator., 6th IFAC Conference on Sensing, Control and Automation Technologies for Agriculture AGRICONTROL 2019., *IFAC-PapersOnLine* 52.30, 2019

[2] Wolf, P.: Evolution of robotic simulators: Using ue 4 to enable real-world quality testing of complex autonomous robots in unstructured environments., *SIMULTECH*. 2020, 2020

[3] Shah, S..: Airsim: High-fidelity visual and physical simulation for autonomous vehicles., *Field and Service Robotics: Results of the 11th International Conference*, Springer International Publishing, 2018

[4] Ganoni, O.: A framework for visually realistic multi-robot simulation in natural environment, *arXiv preprint arXiv:1708.01938*, 2017

[5] Martinez-Gonzalez, P.: Unrealrox: an extremely photorealistic virtual reality environment for robotics simulations and synthetic data generation, Virtual Reality 24, Springer, 2020

[6] MTS Mobile Tiefbau Saugsysteme GmbH.: Anwendungsgebiete, https://www.saugbagger.com/anwendungsgebiete/, 11.04.2024

[7] Goodin, C.: Unmanned ground vehicle simulation with the virtual autonomous navigation environment 2017 International Conference on Military Technologies (ICMT), IEEE, 2017

[8] Körber, M.: Comparing popular simulation environments in the scope of robotics and reinforcement learning, arXiv preprint arXiv:2103.04616, 2021.

[9] Dosovitskiy, A.: CARLA: An open urban driving simulator, Proceedings of the 1st Annual Conference on Robot Learning, PMLR, 2017

[10] Reichardt, M.: Introducing FINROC: A Convenient Real-time Framework for Robotics based on a Systematic Design Approach, Technical Report, Robotics Research Lab, Department of Computer Science, University of Kaiserslautern, 2012

[11] ROS.org. urdf turorials. https://wiki.ros.org/urdf/Tutorials [Accessed: (21.08.2024)].

[12] Vogel, M.: LiDAR-GEiL: LiDAR GPU Exploitation in Lightsimulations, *SIMULTECH*, 2023

[13] Son, Y.: Usefulness of using nvidia isaacsim and isaacgym for ai robot manipulation training, 2023 14th International Conference on Information and Communication Technology Convergence (ICTC), IEEE, 2023

[14] Mittal, M.: Orbit: A unified simulation framework for interactive robot learning environments, IEEE Robotics and Automation Letters, 2023

[15] Unreal Engine 5.1 Documentation. How to set up vehicles,, https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-set-up-vehicles-in-unreal-engine?application_version=5.1, 12.08.2024

[16] Denavit, J.: A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices. Journal of Applied Mechanics, 22(2):215–221, 06 2021.

[17] Szadkowski, K.: Phobos: A tool for creating complex robot models.
Journal of Open Source Software, *Journal of Open Source Software 5.45*, 2020

[18] Rickert, M.: Robotics Library: An object-oriented approach to robot applications. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 733–740, Vancouver, BC, Canada, September 2017.

[19] Felis, M.L.: RBDL: an efficient rigid-body dynamics library using recursive algorithms. Autonomous Robots, pages 1–17, 2016.

[20] Fleischmann, P.: Map-aided off-road path following for autonomous vehicles, Dr.Hut, 2020

[21] Chada, S. K.: Kompakter 6-dof-fahrsimulator für flexible forschungsanwendungen, International Commercial Vehicle Technology Symposium, Springer, 2022

[22] Grondin V.: Training Deep Learning Algorithms on Synthetic Forest Images for Tree Detection, ICRA 2022 Workshop in Innovation in Forestry Robotics: Research and Industry Adoption, 2022.